

UCS503 - Software Engineering

MemoryLane

A Personal Web Memory and Semantic Search System

Submitted by:

1024030716 Rabeeh Ahmad
1024030719 Ishan Singhanian

Group: 3C51 BE Third Year, CSE

Submitted to:

Dr. Raghav B. Venkataramaiyer

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	4
1.1	Background and Context	4
1.2	Problem Statement	4
1.3	Project Objectives	5
2	Methodology and Design	6
2.1	System Architecture	6
2.1.1	Browser Extension	7
2.1.2	Svelte Frontend	7
2.1.3	FastAPI Backend	7
2.1.4	URL Normalization	7
2.2	Content Processing	8
2.2.1	Content Validation	8
2.2.2	YouTube Content Handling	8
2.2.3	Content Chunking	9
2.2.4	Semantic Representation	9
2.3	Information Retrieval	9
2.3.1	Lexical Retrieval	10
2.3.2	Semantic Retrieval	10
2.3.3	Combining Retrieval Signals	10
2.3.4	Candidate Retrieval	10
2.3.5	Webpage-Level Result Aggregation	11
2.4	Technical Stack	11
3	Implementation and Evaluation	12
3.1	Implementation	12
3.1.1	Memory Ingestion Pipeline	12
3.1.2	Search Pipeline	13
3.2	Evaluation Methodology	13
3.3	Evaluation Metrics	14
3.3.1	Mean Reciprocal Rank	14
3.3.2	Recall	14
3.3.3	Normalized Discounted Cumulative Gain	15
3.4	Experimental Configurations	15
3.5	Results	15
3.6	Analysis of Results	15
3.6.1	BM25 Retrieval	16
3.6.2	Semantic Retrieval	16

3.6.3	Combined Retrieval	16
3.7	Comparative Analysis	16
4	Conclusion	18
4.1	Summary of Work	18
4.2	Key Findings	18
4.3	Limitations	19
4.4	Future Work and Recommendations	19
4.5	Final Remarks	20

Introduction

1.1 Background and Context

Modern web browsing involves interacting with a large amount of information across articles, documentation, research papers, tutorials, videos, and other online resources. A user may encounter useful information at one point in time but remember only a general idea of the content when they need it again.

Conventional browser history provides a chronological list of visited URLs, but it does not provide a content-aware way of retrieving previously encountered information. Finding an old resource therefore often requires remembering the website, title, approximate date, or exact keywords that appeared on the page.

MemoryLane is designed to address this problem by treating previously encountered web content as a searchable personal memory. Instead of simply recording that a webpage was visited, the system stores its relevant content and creates a searchable representation of that information.

A user can subsequently issue a natural-language query and retrieve webpages whose stored content is relevant to the query. The system supports both conventional lexical matching and semantic similarity so that useful information can still be found when the wording of the query differs from the wording used on the original webpage.

1.2 Problem Statement

Traditional browser history is primarily concerned with navigation rather than information retrieval. It records information such as URLs and visit history, but it does not provide an effective mechanism for answering content-based questions about previously visited pages.

For example, a user may remember reading something about “methods for reducing database query latency” without remembering the exact terminology or the website on which the information appeared.

A personal web-memory system should therefore be able to:

1. Capture and store useful information from visited webpages.
2. Maintain webpage identity while processing long documents efficiently.
3. Convert webpage content into a representation that supports meaningful search.
4. Retrieve relevant information using natural-language queries.

5. Handle both exact terminology and queries whose wording differs from the stored content.
6. Present results as distinct webpages rather than exposing individual internal text chunks.
7. Provide a practical interface between the browser, processing system, and search engine.

1.3 Project Objectives

The primary objectives of MemoryLane are:

1. **Personal information retrieval:** Develop a system that allows users to search previously encountered web content using natural-language queries.
2. **Web content processing:** Build a pipeline for URL normalization, content validation, chunking, and representation of stored webpages.
3. **Semantic representation:** Generate dense vector representations of webpage content to support meaning-based retrieval.
4. **Efficient search:** Store processed webpage information in a search engine capable of both textual and vector-based retrieval.
5. **Webpage-level results:** Ensure that multiple relevant sections of the same webpage are consolidated into a single user-facing result.
6. **Evaluation:** Measure retrieval performance using established information-retrieval metrics and compare different search configurations.

Methodology and Design

2.1 System Architecture

MemoryLane is organized as a pipeline connecting the browser, user interface, backend processing layer, representation layer, search index, and retrieval system.

The major components are:

1. Browser extension
2. Svelte frontend
3. FastAPI backend
4. URL normalization
5. Content processing and chunking
6. Sentence Transformer embedding model
7. Apache Solr search index
8. Retrieval and ranking module
9. Optional summarization module

The overall ingestion pipeline is:

Browser Page / YouTube Video → Browser Extension → FastAPI → URL Normalization → Content Processing → Chunking → Embeddings → Solr

The search pipeline is:

User Query → Svelte Frontend → FastAPI → Candidate Retrieval → Ranking → Webpage Results

The architecture separates data collection, presentation, processing, storage, and retrieval. This makes individual stages easier to develop, test, and modify independently.

2.1.1 Browser Extension

The browser extension serves as the collection layer of MemoryLane.

When a user encounters a webpage, the extension captures information associated with the page, including its URL, title, and textual content. This information is sent to the backend through the memory-ingestion API.

The extension does not perform the complete search or indexing process itself. Instead, it acts as the interface between the user's browsing environment and the MemoryLane backend.

2.1.2 Svelte Frontend

MemoryLane includes a lightweight frontend built using Svelte, Vite, and JavaScript. The frontend provides the user-facing search interface through which users can submit natural-language queries and inspect retrieved memories.

The interface displays search results with information such as page titles, URLs, and domains. It also supports on-demand summarization of individual results.

The frontend communicates with the FastAPI backend through the Vite development proxy, keeping the presentation layer separate from the backend search and processing logic.

2.1.3 FastAPI Backend

The FastAPI backend provides the primary application interface and coordinates communication between the browser extension, frontend, and search infrastructure.

The main endpoints include:

- `/health` — checks the availability of required backend dependencies.
- `/memory` — accepts webpage information and processes it for storage.
- `/search` — accepts a natural-language query and returns relevant webpages.
- `/summarize` — provides optional summarization functionality.

The backend is responsible for validating incoming data, processing webpage content, generating embeddings, and communicating with Solr.

2.1.4 URL Normalization

URLs can contain variations that still refer to the same logical webpage. For example, differences in capitalization, tracking parameters, URL fragments, default ports, or `www` prefixes can result in multiple representations of the same resource.

MemoryLane normalizes URLs before generating the webpage identifier.

The normalization process includes:

- Normalizing the URL scheme and hostname.
- Removing the `www` prefix.
- Removing default ports.
- Removing URL fragments.
- Removing common tracking parameters.
- Normalizing trailing slashes.
- Sorting query parameters.

A deterministic webpage identifier is generated from the normalized URL. This identifier allows all chunks belonging to one webpage to be associated with the same logical resource.

This is particularly important during retrieval because the system ultimately presents webpages rather than individual chunks.

2.2 Content Processing

2.2.1 Content Validation

Before content is indexed, the backend validates the submitted URL and textual content. Extremely short or invalid content is not treated as a useful webpage memory.

This prevents malformed or insufficient data from unnecessarily occupying the search index.

2.2.2 YouTube Content Handling

MemoryLane also supports storing YouTube videos by using their available transcripts as textual content. For a recognized YouTube video URL, the backend attempts to retrieve the video's transcript rather than treating the video player interface as webpage text.

The retrieved transcript is processed using the same pipeline as normal webpage content. It is divided into chunks, converted into dense representations, and stored in Solr. This allows information contained within educational or informational videos to become searchable alongside conventional webpages.

2.2.3 Content Chunking

Webpages can contain substantially more text than can be effectively represented by a single vector. MemoryLane therefore divides webpage content into smaller chunks.

The current configuration uses a minimum chunk size of approximately 300 words and a maximum of approximately 500 words.

Chunking provides two important advantages. First, it allows different sections of a long webpage to be represented independently. Second, retrieval can identify the particular section that is most relevant to a query.

Each chunk retains metadata connecting it back to its original webpage.

The stored information includes:

- Webpage identifier
- URL
- Domain
- Page title
- Chunk identifier
- Total number of chunks
- Timestamp
- Dense embedding

2.2.4 Semantic Representation

Each text chunk is converted into a dense vector using the `all-MiniLM-L6-v2` Sentence Transformer model.

The model produces a 384-dimensional representation for each chunk. The resulting vectors encode semantic characteristics of the text and allow related concepts to be compared using vector similarity.

Embeddings are normalized before similarity-based retrieval, and cosine similarity is used for the dense-vector search.

2.3 Information Retrieval

MemoryLane uses more than one retrieval signal because different queries benefit from different forms of matching.

2.3.1 Lexical Retrieval

The first retrieval mechanism is BM25-based lexical search.

BM25 is useful when a query contains specific words, names, technical identifiers, or phrases that directly occur in the stored content.

MemoryLane uses Solr's eDisMax query mechanism with the `text` field as the primary field and the title field given additional importance.

Lexical retrieval therefore provides a strong mechanism for queries where the exact terminology is informative.

2.3.2 Semantic Retrieval

The second retrieval mechanism uses the dense representations generated by the Sentence Transformer model.

When a user submits a query, the query is converted into the same 384-dimensional embedding space as the stored chunks. Solr then performs K-nearest-neighbour retrieval to identify semantically similar content.

This allows MemoryLane to retrieve information when the user's query and the original webpage use different words but describe a similar concept.

2.3.3 Combining Retrieval Signals

The system can use the lexical and semantic retrieval signals together.

The scores generated by the two retrieval methods are first normalized independently using min-max normalization:

$$s'_i = \frac{s_i - s_{\min}}{s_{\max} - s_{\min}}$$

The current production configuration assigns equal weights to the two signals:

$$S = 0.5S_{\text{BM25}} + 0.5S_{\text{vector}}$$

This combined ranking is used by the main search pipeline, while the individual retrieval configurations can also be evaluated independently.

The purpose of combining the signals is not to replace either retrieval method, but to provide the search system with additional evidence when ranking results.

2.3.4 Candidate Retrieval

For a search query, the system retrieves a candidate pool from the available retrieval mechanisms.

The current configuration retrieves up to 30 candidates from each retrieval source before ranking the final results.

This candidate-generation stage allows the system to consider a larger set of potentially relevant chunks than the number of results ultimately presented to the user.

2.3.5 Webpage-Level Result Aggregation

The internal search process operates on chunks, but presenting chunks directly would cause the same webpage to potentially appear multiple times.

MemoryLane therefore groups retrieved chunks using their `webpage_id`.

For each webpage, the highest-scoring retrieved chunk is used to represent that webpage’s relevance. The webpages are then ranked and returned as unique results.

The process can therefore be represented as:

**Chunk Retrieval → Webpage Grouping → Best-Chunk Selection →
Webpage Ranking**

This maintains the advantages of fine-grained chunk retrieval while producing a cleaner user-facing search experience.

2.4 Technical Stack

Component	Technology
Programming language	Python
Frontend	Svelte / Vite / JavaScript
Backend framework	FastAPI
Search engine	Apache Solr 9.7
Lexical retrieval	BM25 / Solr eDisMax
Vector retrieval	Solr KNN search
Embedding model	Sentence Transformers <code>all-MiniLM-L6-v2</code>
Embedding dimension	384
Containerization	Docker
Communication	HTTP / JSON
Optional summarization	Gemini-based summarization

Table 2.1: MemoryLane Technical Stack

Implementation and Evaluation

3.1 Implementation

The backend is divided into modular components so that different responsibilities remain separated.

The major implementation modules are:

- `main.py` — API endpoints and application orchestration.
- `models.py` — request and response models.
- `url_utils.py` — URL normalization and webpage identifier generation.
- `retrieval.py` — candidate retrieval, score processing, ranking, and webpage aggregation.
- `solr_client.py` — communication with the Apache Solr index.
- `config.py` — system configuration and retrieval parameters.

3.1.1 Memory Ingestion Pipeline

When a webpage or supported YouTube video is submitted to the `/memory` endpoint, the following sequence is performed:

1. Validate the supplied URL.
2. Normalize the URL.
3. Generate a deterministic webpage identifier.
4. Check whether the webpage already exists.
5. Validate the supplied content.
6. For supported YouTube URLs, retrieve the available transcript and use it as the textual content.
7. Divide the content into chunks.
8. Generate embeddings for the chunks.

9. Remove outdated chunks associated with the same webpage when required.
10. Store the processed chunks and metadata in Solr.

This provides a consistent ingestion path for normal webpages, supported YouTube videos, and evaluation data.

3.1.2 Search Pipeline

A search request follows the following sequence:

1. Receive the natural-language query.
2. Retrieve lexical candidates.
3. Generate a dense representation of the query.
4. Retrieve semantically similar candidates.
5. Combine the available candidate results.
6. Process their retrieval scores.
7. Group chunks by webpage.
8. Select the strongest representation of each webpage.
9. Return the requested number of unique webpages.

3.2 Evaluation Methodology

The retrieval component was evaluated using the SciFact information-retrieval dataset.

The benchmark was adapted to the MemoryLane data model so that the documents could pass through the actual ingestion pipeline.

The evaluation setup contained:

- 1,000 unique documents
- 100 evaluation queries
- 106 relevant documents represented in the selected relevance data

The documents were indexed using the actual MemoryLane `/memory` endpoint rather than being inserted directly into Solr.

Therefore, the benchmark exercised the application's real processing pipeline, including URL handling, content processing, chunking, embedding generation, and indexing.

The benchmark documents use synthetic URLs of the form:

`https://scifact.org/doc/{document_id}`

These URLs provide unique webpage identifiers for the benchmark. The actual document titles and scientific content originate from SciFact.

Consequently, this experiment evaluates the information retrieval component under a controlled corpus rather than evaluating live web crawling or webpage extraction.

3.3 Evaluation Metrics

Three retrieval metrics were used to measure the quality of the returned rankings.

3.3.1 Mean Reciprocal Rank

Mean Reciprocal Rank at ten (MRR@10) measures how early the first relevant result appears among the top ten results.

For a query whose first relevant result occurs at rank r :

$$RR = \frac{1}{r}$$

The average reciprocal rank across all queries produces MRR.

A higher value indicates that relevant information tends to appear earlier in the search results.

3.3.2 Recall

Recall@10 measures the proportion of relevant documents that are retrieved within the first ten results:

$$Recall@10 = \frac{\text{Relevant documents retrieved in top 10}}{\text{Total relevant documents}}$$

A higher value indicates that fewer relevant documents are missed by the retrieval system.

3.3.3 Normalized Discounted Cumulative Gain

nDCG@10 evaluates the quality of the ordering of the retrieved results, giving greater importance to relevant results that appear near the top.

Binary relevance was used for this experiment, meaning that a document was treated as either relevant or non-relevant.

3.4 Experimental Configurations

Three retrieval configurations were evaluated:

1. **BM25 Only:** Uses lexical retrieval without the semantic retrieval contribution.
2. **Vector Only:** Uses dense semantic retrieval without the lexical contribution.
3. **Combined Retrieval:** Uses the production configuration in which lexical and semantic retrieval signals are combined with equal weighting.

Each configuration produced the top ten webpage-level results for each of the 100 evaluation queries.

3.5 Results

The evaluation results are shown in Table 3.1.

Method	MRR@10	Recall@10	nDCG@10
BM25 Only	0.6482	0.8117	0.6776
Vector Only	0.7795	0.8867	0.8008
Combined Retrieval	0.7538	0.9167	0.7890

Table 3.1: SciFact Retrieval Evaluation Results

3.6 Analysis of Results

The results show that the different retrieval approaches have different strengths on the selected benchmark.

3.6.1 BM25 Retrieval

BM25 achieved an MRR@10 of 0.6482, Recall@10 of 0.8117, and nDCG@10 of 0.6776.

This provides a useful lexical baseline. Its performance demonstrates that conventional term-based retrieval is capable of retrieving a substantial portion of the relevant documents.

However, scientific queries may describe concepts using language that differs from the exact wording in the relevant documents. This can limit purely lexical retrieval.

3.6.2 Semantic Retrieval

Vector retrieval achieved the highest MRR@10, at 0.7795, and the highest nDCG@10, at 0.8008.

This indicates that semantic representations were effective at identifying relevant scientific content and placing relevant documents near the top of the ranking.

The result is particularly relevant to MemoryLane because users may remember the meaning of information they encountered without remembering the exact terminology used on the original webpage.

3.6.3 Combined Retrieval

The combined configuration achieved the highest Recall@10, with a value of 0.9167.

This indicates that incorporating both lexical and semantic evidence allowed the system to retrieve a larger proportion of the relevant documents within the top ten.

At the same time, vector-only retrieval achieved slightly higher MRR@10 and nDCG@10. This shows that the best retrieval configuration can depend on the characteristics of the underlying documents and queries.

The current equal weighting therefore represents a practical configuration rather than a claim that equal weighting is universally optimal.

3.7 Comparative Analysis

The evaluation can be summarized as follows:

Observation	Interpretation
Highest MRR@10	Vector retrieval places the first relevant result highest on average.
Highest Recall@10	The combined configuration retrieves the largest proportion of relevant documents within the top ten.
Highest nDCG@10	Vector retrieval provides the strongest ranking quality on the selected benchmark.
BM25 baseline	Lexical retrieval remains effective when query terminology matches the stored content.

Table 3.2: Comparison of Retrieval Configurations

These results demonstrate why MemoryLane supports multiple retrieval signals. Different search strategies provide different forms of evidence, and their performance can vary depending on the query and corpus.

Conclusion

4.1 Summary of Work

MemoryLane was developed as a personal web-memory system that allows previously encountered web content to be stored, processed, indexed, and retrieved through natural-language search.

The system provides a complete pipeline beginning with browser content collection and continuing through URL normalization, content chunking, semantic representation, search indexing, and result ranking.

The system also supports storing YouTube video content through available transcripts, allowing video information to enter the same processing and retrieval pipeline as conventional webpages.

The user-facing interface is implemented using Svelte, Vite, and JavaScript, while the backend is implemented using FastAPI. Apache Solr provides the underlying search infrastructure. Webpage and transcript chunks are represented using 384-dimensional embeddings generated by the `all-MiniLM-L6-v2` model.

The retrieval layer supports both conventional lexical search and semantic vector search. These approaches can also be used together to provide a broader set of retrieval evidence.

4.2 Key Findings

The evaluation produced several important findings.

1. **MemoryLane successfully performs content-based retrieval.** The system was able to process and index a controlled corpus of 1,000 documents and evaluate retrieval over 100 queries.
2. **Semantic representations were highly effective.** Vector retrieval achieved the highest MRR@10 of 0.7795 and nDCG@10 of 0.8008.
3. **Multiple retrieval signals improved coverage.** The combined configuration achieved the highest Recall@10 of 0.9167, showing that different retrieval mechanisms can complement one another.
4. **Retrieval quality depends on the evaluation objective.** The configuration with the highest recall was not the same as the configuration with the highest MRR and nDCG. This highlights the importance of evaluating search systems using multiple metrics rather than relying on a single score.

5. **Apache Solr provided an efficient search infrastructure.** The use of Apache Solr allowed lexical retrieval, vector retrieval, indexing, and ranking to be handled within a unified search platform, providing responsive retrieval during practical system testing.

4.3 Limitations

The current implementation of MemoryLane has several project-level limitations.

First, the quality of stored memories depends on the quality and availability of the extracted webpage content. Dynamic webpages, pages with limited accessible text, and YouTube videos without an available transcript may not always provide sufficient content for storage.

Second, the current system is primarily designed as a personal web-memory prototype. Its storage and retrieval architecture has not yet been extensively tested at the scale of very large personal browsing histories.

Third, when the optional Gemini API is used for summarization, personal browsing content must be sent to an external service. This introduces privacy concerns because sensitive information or identifiable browsing fingerprints could potentially be exposed.

Finally, the current user interface provides the core search and result-viewing functionality, but additional usability features would be required for a more complete history-management experience.

4.4 Future Work and Recommendations

Several extensions can improve MemoryLane in future versions.

1. **Improved retrieval ranking:** Investigate alternative ranking and score-combination strategies to improve search quality across different query types.
2. **Improved chunking:** Explore sentence-aware, paragraph-aware, and overlapping chunking strategies.
3. **Embedding models:** Compare the current embedding model with newer retrieval-oriented models.
4. **Reranking:** Introduce a dedicated reranking stage for the strongest candidate results.
5. **Larger-scale evaluation:** Test the system on larger and more diverse information retrieval datasets.

6. **Scalability:** Study indexing and retrieval performance as the number of stored webpages increases substantially.
7. **Recent browsing history integration:** Extend MemoryLane to maintain a recent history of accessed URLs alongside stored content memories. This would provide a more holistic browsing-history experience by combining conventional history features with content-aware retrieval.
8. **Privacy and local processing:** Investigate local-first processing and storage approaches so that personal browsing information can remain under the user's control.

4.5 Final Remarks

MemoryLane demonstrates how browser history can be extended from a simple record of visited URLs into a searchable memory of previously encountered information.

By storing webpage content, processing it into searchable representations, and providing both lexical and semantic retrieval capabilities, the system allows users to search for information based on what they remember rather than only what they can recall about the original webpage.

The evaluation demonstrates strong retrieval performance on the selected scientific benchmark, while also showing that different retrieval approaches have different strengths. Overall, the project establishes a foundation for a practical personal information-retrieval system that can be further improved through better ranking, representation, scalability, and real-world evaluation.